FACULDADE FIAP
Curso: CIENCIA DA COMPUTAÇÃO

SISTEMA DE VERIFICAÇÃO DE PRÉ-DECOLAGEM

Aluno: Emanuel Domingues

São Paulo - SP
2026

SUMÁRIO

1. Introdução 3

2. Organização da telemetria e faixas de decisão 3

2.1 Estrutura dos dados coletados 4

3. Algoritmo de verificação 4

3.1 Pseudocódigo 4

3.2 Fluxograma da decisão 5

4. Explicação detalhada do código 6

4.1 Importações e dicionário LIMITES 6

4.2 Função verificar_dados() 6

4.3 Função analise_energetica() 7

4.4 Função analise_ia() 7

4.5 Funções de saída: relatório e planilha 8

5. Análise dos cenários simulados 9

5.1 Print simulado da execução aprovada 9

5.2 Print simulado da execução crítica 9

6. Análise energética 10

7. Análise assistida por IA 11

8. Pontos fortes e oportunidades de melhoria 11

9. Reflexão crítica: ética, impacto social e sustentabilidade 12

10. Critérios de avaliação atendidos 12

11. Conclusão 12

1. Introdução

Este documento apresenta uma análise técnica detalhada do script Python entregue para a atividade integradora da Missão Aurora. O programa coleta dados de telemetria, valida faixas seguras de operação, calcula autonomia energética, realiza uma triagem assistida por IA e emite a decisão final de voo: PRONTO PARA DECOLAR ou DECOLAGEM ABORTADA.

A organização do relatório segue o padrão acadêmico do arquivo-modelo: capa temática, sumário, seções hierárquicas, análise da telemetria, algoritmo de decisão, explicação do script, análise energética, reflexão crítica e critérios de avaliação.

Resumo executivo	
Propósito	Automatizar a checagem de segurança da nave antes da decolagem.
Entradas	Temperaturas, integridade estrutural, energia, pressões e status dos módulos críticos.
Saídas	Alertas, decisão operacional, análise energética, análise assistida por IA e planilha Excel.

2. Organização da telemetria e faixas de decisão

O enunciado da atividade exige a interpretação de temperatura interna e externa, integridade estrutural, nível de energia, pressão dos tanques e status de módulos críticos. O script implementa exatamente esse conjunto de variáveis em um dicionário chamado d a d o s , reunindo valores numéricos e estados booleanos.

Parâmetro	Faixa ou regra adotada no código	Impacto operacional
Temperatura interna	-10 °C a 40 °C	Avalia condição interna da nave e risco térmico aos circuitos.
Temperatura externa	-80 °C a 60 °C	Verifica se o ambiente externo está dentro da faixa suportada.
Integridade estrutural	Deve ser igual a 1	Qualquer falha compromete a segurança do lançamento.
Nível de energia	Mínimo de 85%	Garante margem mínima de disponibilidade energética.
Pressão dos tanques	200 kPa a 350 kPa	Evita insuficiência de alimentação ou sobrepressão.
Módulos críticos	Todos devem estar em estado verdadeiro	A ausência de redundância total leva ao aborto da missão.

Tabela 1 - Organização da telemetria implementada pelo script.

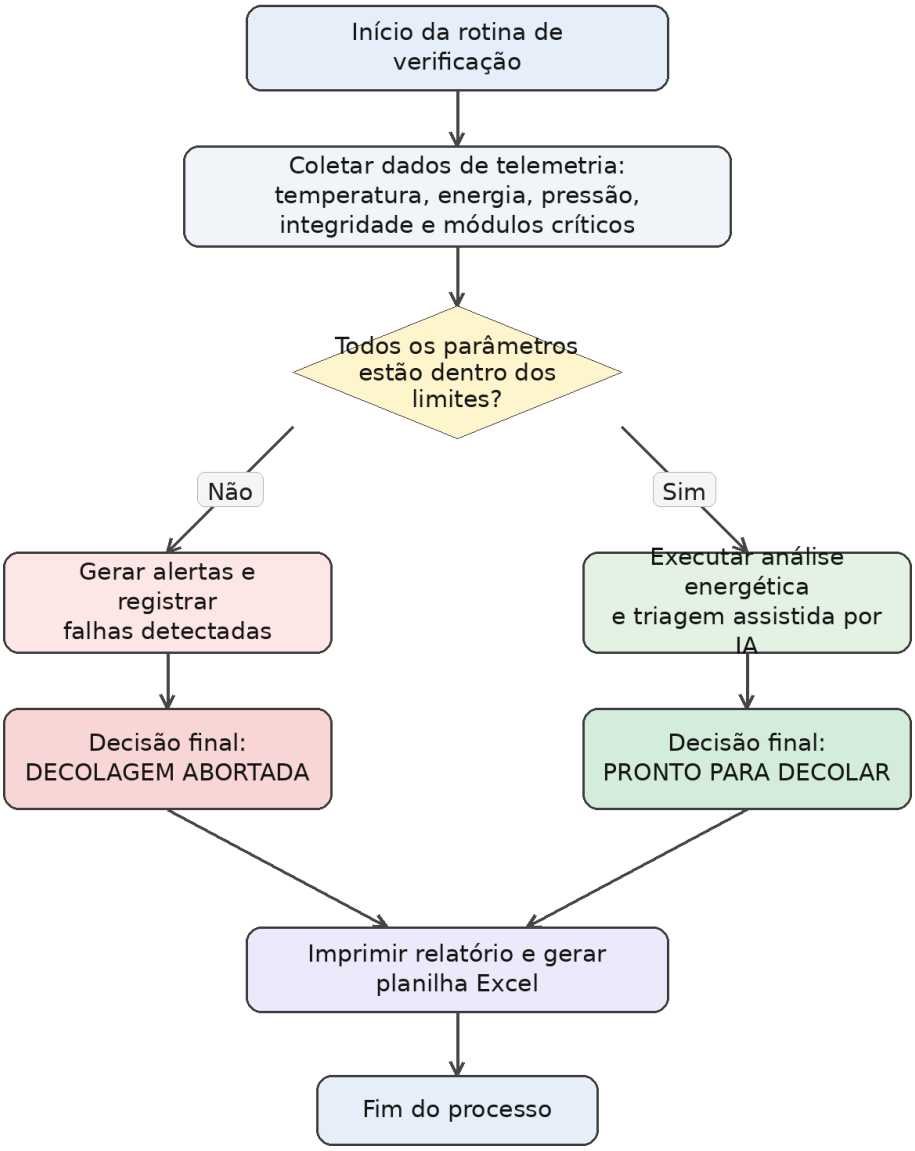
2.1 Estrutura dos dados coletados

A função `coletar_dados()` solicita os valores ao usuário, converte cada entrada para o tipo apropriado e agrupa os módulos críticos em um subdicionário. Essa modelagem é adequada porque mantém a telemetria organizada e pronta para as etapas seguintes.

```python
def coletar_dados():
    dados = {
        "timestamp": datetime.now().isoformat(),
        "temp_interna": float(input("Digite a temperatura interna (°C): ")),
        "temp_externa": float(input("Digite a temperatura externa (°C): ")),
        "integridade_estrutural": int(input("Digite a integridade estrutural (1 para OK, 0 para falha): ")),
        "nivel_energia": float(input("Digite o nível de energia (%): ")),
        "pressao_tanque_ox": float(input("Digite a pressão do tanque de oxigênio (kPa): ")),
        "pressao_tanque_comb": float(input("Digite a pressão do tanque de combustível (kPa): ")),
        "modulos": {
            "propulsao": bool(int(input("Digite o status do módulo de propulsão (1 para OK, 0 para falha): "))),
            "navegacao": bool(int(input("Digite o status do módulo de navegação (1 para OK, 0 para falha): "))),
            "comunicacao": bool(int(input("Digite o status do módulo de comunicação (1 para OK, 0 para falha): "))),
            "controle_termico": bool(int(input("Digite o status do módulo de controle térmico (1 para OK, 0 para falha): "))),
            "separacao_estagios": bool(int(input("Digite o status da separação de estágios (1 para OK, 0 para falha): ")))
        }
    }
    return dados
```

2.2 Fluxograma da decisão

Fluxograma lógico da decisão de decolagem



3. Explicação detalhada do código

A seguir, o script é destrinchado por blocos, com foco no papel de cada função, no raciocínio lógico e no valor técnico de cada parte do programa.

Bloco	Linhas	Função no sistema
Importações e Limites	s1-13	Declaram bibliotecas e centralizam os limites operacionais.
coletar_dados()	20-37	Converte entradas do usuário em telemetria estruturada.
verificar_dados()	40-96	Aplica regras de segurança e consolida a decisão.
analise_energetica()	99-118	Calcula consumo real e autonomia disponível.
analise_ia()	121-178	Classifica a situação e produz recomendações.
gerar_relatorio()	181-231	Monta a saída textual legível no terminal.
gerar_planilha()	234-311	Documenta os resultados em múltiplas abas Excel.
Fluxo principal	314-320	Encadeia as funções na ordem correta da missão.

Tabela 2 - Mapa de leitura do script por blocos.

Importações e dicionário LIMITES

O script inicia com a importação das bibliotecas necessárias e também define os parâmetros de limite para o foguete.

```python
import pandas as pd
import random
import json
from datetime import datetime


LIMITES = {
    "temp_interna_min": -10,
    "temp_interna_max": 40,
    "temp_externa_min": -80,
    "temp_externa_max": 60,
    "integridade_estrutural": 1,
    "nivel_energia_min": 85,
    "pressao_tanque_min": 200,
    "pressao_tanque_max": 350,
    "modulos_criticos_ok": True,
}
```

Função verificar_dados()

Logo após coletar os dados do foguete, ele inicia a verificação dos dados

```python
def verificar_dados(dados):
    alertas = []
    verificacoes = {}

    ti = dados["temp_interna"]
    ok_ti = LIMITES["temp_interna_min"] <= ti <= LIMITES["temp_interna_max"]
    verificacoes["temp_interna"] = ok_ti
    if not ok_ti:
        alertas.append(f"[ALERTA] Temperatura interna fora do limite: {ti}°C "
                       f"(esperado: {LIMITES['temp_interna_min']} a {LIMITES['temp_interna_max']}°C)")

    te = dados["temp_externa"]
    ok_te = LIMITES["temp_externa_min"] <= te <= LIMITES["temp_externa_max"]
    verificacoes["temp_externa"] = ok_te
    if not ok_te:
        alertas.append(f"[ALERTA] Temperatura externa fora do limite: {te}°C "
                       f"(esperado: {LIMITES['temp_externa_min']} a {LIMITES['temp_externa_max']}°C)")

    ie = dados["integridade_estrutural"]
    ok_ie = ie == LIMITES["integridade_estrutural"]
    verificacoes["integridade_estrutural"] = ok_ie
    if not ok_ie:
        alertas.append("[CRÍTICO] Integridade estrutural comprometida! (valor=0)")

    ne = dados["nivel_energia"]
    ok_ne = ne >= LIMITES["nivel_energia_min"]
    verificacoes["nivel_energia"] = ok_ne
    if not ok_ne:
        alertas.append(f"[ALERTA] Nível de energia insuficiente: {ne}% "
                       f"(mínimo: {LIMITES['nivel_energia_min']}%)")

    po = dados["pressao_tanque_ox"]
    pc = dados["pressao_tanque_comb"]
    ok_po = LIMITES["pressao_tanque_min"] <= po <= LIMITES["pressao_tanque_max"]
    ok_pc = LIMITES["pressao_tanque_min"] <= pc <= LIMITES["pressao_tanque_max"]
    verificacoes["pressao_tanque_ox"] = ok_po
    verificacoes["pressao_tanque_comb"] = ok_pc
    if not ok_po:
        alertas.append(f"[CRÍTICO] Pressão tanque oxidante fora do limite: {po} kPa")
    if not ok_pc:
        alertas.append(f"[CRÍTICO] Pressão tanque combustível fora do limite: {pc} kPa")

    modulos_falhos = [m for m, status in dados["modulos"].items() if not status]
    ok_modulos = len(modulos_falhos) == 0
    verificacoes["modulos_criticos"] = ok_modulos
```

```python
    todos_ok = all(verificacoes.values())
    decisao = "PRONTO PARA DECOLAR" if todos_ok else "DECOLAGEM ABORTADA"

    return {
        "verificacoes": verificacoes,
        "alertas": alertas,
        "decisao": decisao,
        "aprovado": todos_ok
    }
```

3.1 Função analise_energetica()

A análise energética trabalha com uma capacidade total fixa de 500 kWh, consumo de decolagem de 120 kWh e fator de perdas de 8%. O cálculo é consistente e diferencia limiar mínimo operacional de suficiência física.

```python
def analise_energetica(nivel_energia_pct):
    capacidade_total_kwh = 500.0
    consumo_decolagem_kwh = 120.0
    fator_perdas = 0.08

    carga_atual_kwh = capacidade_total_kwh * (nivel_energia_pct / 100)
    perdas_kwh = consumo_decolagem_kwh * fator_perdas
    consumo_real = consumo_decolagem_kwh + perdas_kwh
    autonomia_kwh = carga_atual_kwh - consumo_real
    suficiente = autonomia_kwh > 0

    return {
        "capacidade_total_kwh": capacidade_total_kwh,
        "carga_atual_kwh": round(carga_atual_kwh, 2),
        "consumo_decolagem_kwh": consumo_decolagem_kwh,
        "perdas_kwh": round(perdas_kwh, 2),
        "consumo_real_kwh": round(consumo_real, 2),
        "autonomia_pos_decolagem_kwh": round(autonomia_kwh, 2),
        "suficiente": suficiente
    }
```

3.2 Função analise_ia()

Construi um sistema de analise baseada em regras. Ela consegue: classificar os dados, destaca anomalias e propõe ações de mitigação.

```python
def analise_ia(dados, resultado_verificacao, energia):
    classificacoes = []
    anomalias = []
    sugestoes = []

    ti = dados["temp_interna"]
    if ti < 0:
        classificacoes.append("Temperatura interna: FRIA (risco de condensação em circuitos)")
        anomalias.append("Temperatura interna abaixo de zero — risco de falha em sensores")
        sugestoes.append("Ativar sistema de aquecimento interno antes da decolagem")
    elif ti <= 30:
        classificacoes.append("Temperatura interna: NOMINAL")
    else:
        classificacoes.append("Temperatura interna: ELEVADA (monitorar resfriamento)")
        sugestoes.append("Verificar sistema de refrigeração interno")

    ne = dados["nivel_energia"]
    if ne >= 95:
        classificacoes.append("Energia: EXCELENTE")
    elif ne >= 85:
        classificacoes.append("Energia: ADEQUADA")
    else:
        classificacoes.append("Energia: INSUFICIENTE — risco operacional")
        anomalias.append("Nível de energia abaixo do mínimo recomendado")
        sugestoes.append("Recarregar bancos de energia antes da tentativa de decolagem")

    po = dados["pressao_tanque_ox"]
    pc = dados["pressao_tanque_comb"]
    if po < 210 or pc < 210:
        anomalias.append("Pressão em tanques próxima ao limite inferior — risco de falha de propulsão")
        sugestoes.append("Pressurizar tanques ao nível nominal (270–300 kPa)")
    elif po > 330 or pc > 330:
        anomalias.append("Pressão em tanques próxima ao limite superior — risco de ruptura")
        sugestoes.append("Liberar pressão excedente via válvula de alívio")
    else:
        classificacoes.append("Pressão dos tanques: NOMINAL")

    falhos = [m for m, s in dados["modulos"].items() if not s]
    if falhos:
        anomalias.append(f"Módulo(s) crítico(s) com falha detectada: {', '.join(falhos)}")
        sugestoes.append("Executar diagnóstico completo e substituir módulo(s) defeituoso(s)")
    else:
        classificacoes.append("Todos os módulos críticos: OPERACIONAIS")
```

```python
    if dados["integridade_estrutural"] == 0:
        anomalias.append("Integridade estrutural comprometida — possível dano físico à estrutura")
        sugestoes.append("Inspeção visual e ultrassônica completa antes de qualquer nova tentativa")

    if not resultado_verificacao["aprovado"]:
        sugestoes.append("Missão deve ser adiada até resolução de todos os alertas críticos")
    else:
        sugestoes.append("Todos os parâmetros nominais — missão pode prosseguir conforme planejado")

    return {
        "classificacoes": classificacoes,
        "anomalias": anomalias if anomalias else ["Nenhuma anomalia detectada"],
        "sugestoes": sugestoes
    }
```

4.4 Função gerar_relatorio()

Nessa parte , ele coleta todos os dados armazenas em arrays e listas e gera o relatorio final , para a tomada da decisão

```
def gerar_relatorio(dados, resultado, energia, ia):
    sep = "=" * 60
    print(f"\n{sep}")
    print(" RELATORIO DE VERIFICAÇÃO")
    print(f"  {dados['timestamp']}")
    print(sep)

    print("\n DADOS COLETADOS")
    print(f"  Temperatura Interna  : {dados['temp_interna']} °C")
    print(f"  Temperatura Externa  : {dados['temp_externa']} °C")
    print(f"  Integridade Estrut.  : {'OK (1)' if dados['integridade_estrutural'] else 'FALHA (0)'}")
    print(f"  Nível de Energia     : {dados['nivel_energia']} %")
    print(f"  Pressão Tanque Ox.   : {dados['pressao_tanque_ox']} kPa")
    print(f"  Pressão Tanque Comb. : {dados['pressao_tanque_comb']} kPa")
    print("  Módulos Críticos:")
    for modulo, status in dados["modulos"].items():
        print(f"   {modulo.replace('_', ' ').title()} : {'OK (1)' if status else 'FALHA (0)'}")

    print("\nVERIFICAÇÕES DE SEGURANÇA")
    for check, ok in resultado["verificacoes"].items():
        print(f"  {check.replace('_', ' ').title()} : {'OK' if ok else 'FALHA'}")

    if resultado["alertas"]:
        print("\n ALERTAS")
        for a in resultado["alertas"]:
            print(f"  {a}")

    print("\n ANÁLISE ENERGÉTICA")
    e = energia
    print(f"  Capacidade Total     : {e['capacidade_total_kwh']} kWh")
    print(f"  Carga Atual          : {e['carga_atual_kwh']} kWh")
    print(f"  Consumo Decolagem    : {e['consumo_decolagem_kwh']} kWh")
    print(f"  Perdas Estimadas     : {e['perdas_kwh']} kWh")
    print(f"  Consumo Real Total   : {e['consumo_real_kwh']} kWh")
    print(f"  Autonomia Pós-Dec.   : {e['autonomia_pos_decolagem_kwh']} kWh  "
          f"{'Suficiente' if e['suficiente'] else 'Insuficiente'}")

    print("\n ANÁLISE DE IA")
    classificacoes = ia.get("classificacoes", []) if isinstance(ia, dict) else []
    anomalias = ia.get("anomalias", []) if isinstance(ia, dict) else []
    sugestoes = ia.get("sugestoes", []) if isinstance(ia, dict) else []
```

```python
print("  Classificações:")
if classificacoes:
    for c in classificacoes:
        print(f"    • {c}")
else:
    print("    • Nenhuma classificação disponível")

print("  Anomalias:")
if anomalias:
    for a in anomalias:
        print(f"    ⚠ {a}")
else:
    print("    ⚠ Nenhuma anomalia detectada")

print("  Sugestões de Risco:")
if sugestoes:
    for s in sugestoes:
        print(f"    → {s}")
else:
    print("    → Nenhuma sugestão disponível")
```

4. Análise dos cenários simulados

Para demonstrar o comportamento do programa, foram simulados dois conjuntos de entrada: um cenário nominal, em que a missão é aprovada, e um cenário crítico, em que a decolagem é abortada.

4.1 Print simulado DECOLAGEM ABORTADA

Abaixo mostra um exemplo de quando o decolagem é abortada

```
============================================================
 RELATORIO DE VERIFICAÇÃO
  2026-03-30T09:50:31.337582
============================================================

 DADOS COLETADOS
  Temperatura Interna  : 30.0 °C
  Temperatura Externa  : 12.0 °C
  Integridade Estrut.  : OK (1)
  Nível de Energia     : 45.0 %
  Pressão Tanque Ox.   : 52.0 kPa
  Pressão Tanque Comb. : 12.0 kPa
  Módulos Críticos:
   Propulsao : FALHA (0)
   Navegacao : OK (1)
   Comunicacao : OK (1)
   Controle Termico : OK (1)
   Separacao Estagios : FALHA (0)

VERIFICAÇÕES DE SEGURANÇA
  Temp Interna : OK
  Temp Externa : OK
  Integridade Estrutural : OK
  Nivel Energia : FALHA
  Pressao Tanque Ox : FALHA
  Pressao Tanque Comb : FALHA
  Modulos Criticos : FALHA
```

```
ALERTAS
 [ALERTA] Nível de energia insuficiente: 45.0% (mínimo: 85%)
 [CRÍTICO] Pressão tanque oxidante fora do limite: 52.0 kPa
 [CRÍTICO] Pressão tanque combustível fora do limite: 12.0 kPa
 [CRÍTICO] Módulos com falha: propulsao, separacao_estagios

ANÁLISE ENERGÉTICA
 Capacidade Total     : 500.0 kWh
 Carga Atual          : 225.0 kWh
 Consumo Decolagem    : 120.0 kWh
 Perdas Estimadas     : 9.6 kWh
 Consumo Real Total   : 129.6 kWh
 Autonomia Pós-Dec.   : 95.4 kWh  Suficiente

ANÁLISE DE IA
 Classificações:
   • Temperatura interna: NOMINAL
   • Energia: INSUFICIENTE — risco operacional
 Anomalias:
   ⚠ Nível de energia abaixo do mínimo recomendado
   ⚠ Pressão em tanques próxima ao limite inferior — risco de falha de propulsão
   ⚠ Módulo(s) crítico(s) com falha detectada: propulsao, separacao_estagios
 Sugestões de Risco:
   → Recarregar bancos de energia antes da tentativa de decolagem
   → Pressurizar tanques ao nível nominal (270–300 kPa)
   → Executar diagnóstico completo e substituir módulo(s) defeituoso(s)
   → Missão deve ser adiada até resolução de todos os alertas críticos
```


```
 Sugestões de Risco:
   → Recarregar bancos de energia antes da tentativa de decolagem
   → Pressurizar tanques ao nível nominal (270–300 kPa)
   → Executar diagnóstico completo e substituir módulo(s) defeituoso(s)
   → Missão deve ser adiada até resolução de todos os alertas críticos
```


```
 DECISÃO FINAL: DECOLAGEM ABORTADA
```

4.2 Print simulado DECOLAGEM AUTORIZADA

```
============================================================
 RELATORIO DE VERIFICAÇÃO
  2026-03-30T10:20:01.655311
============================================================

 DADOS COLETADOS
  Temperatura Interna  : 30.0 °C
  Temperatura Externa  : 50.0 °C
  Integridade Estrut.  : OK (1)
  Nível de Energia     : 100.0 %
  Pressão Tanque Ox.   : 270.0 kPa
  Pressão Tanque Comb. : 200.0 kPa
  Módulos Críticos:
   Propulsao : OK (1)
   Navegacao : OK (1)
   Comunicacao : OK (1)
   Controle Termico : OK (1)
   Separacao Estagios : OK (1)

VERIFICAÇÕES DE SEGURANÇA
  Temp Interna : OK
  Temp Externa : OK
  Integridade Estrutural : OK
  Nivel Energia : OK
  Pressao Tanque Ox : OK
  Pressao Tanque Comb : OK
  Modulos Criticos : OK
```

```
ANÁLISE ENERGÉTICA
 Capacidade Total     : 500.0 kWh
 Carga Atual          : 500.0 kWh
 Consumo Decolagem    : 120.0 kWh
 Perdas Estimadas     : 9.6 kWh
 Consumo Real Total   : 129.6 kWh
 Autonomia Pós-Dec.   : 370.4 kWh  Suficiente

ANÁLISE DE IA
 Classificações:
   • Temperatura interna: NOMINAL
   • Energia: EXCELENTE
   • Todos os módulos críticos: OPERACIONAIS
 Anomalias:
   ⚠ Pressão em tanques próxima ao limite inferior — risco de falha de propulsão
 Sugestões de Risco:
   → Pressurizar tanques ao nível nominal (270–300 kPa)
   → Todos os parâmetros nominais — missão pode prosseguir conforme planejado

____________________________________________________________

 DECISÃO FINAL: PRONTO PARA DECOLAR
____________________________________________________________
```

Figura 3 - Saída simulada do script em um cenário com aborto da missão.

5. Análise energética

No cenário aprovado, a carga atual é 500 × 0,92 = 460,0 kWh. As perdas são calculadas como 120 × 0,08 = 9,6 kWh. Portanto, o consumo real de decolagem é 129,6 kWh, restando 330,4 kWh de autonomia após a manobra.

No cenário crítico, apesar de a energia estar abaixo do mínimo recomendado de 85%, a autonomia física ainda é positiva: 500 × 0,80 = 400,0 kWh; consumo real = 129,6 kWh; autonomia remanescente = 270,4 kWh. Isso mostra que o código adota uma regra operacional mais conservadora que a simples suficiência matemática.

Métrica	Cenário A	Cenário B
Capacidade total	500,0 kWh	500,0 kWh
Carga atual	460,0 kWh	400,0 kWh
Consumo de decolagem	120,0 kWh	120,0 kWh
Perdas	9,6 kWh	9,6 kWh
Consumo real	129,6 kWh	129,6 kWh
Autonomia pós-decolagem	330,4 kWh	270,4 kWh
Suficiência física	Sim	Sim
Aprovação operacional	Sim	Não

6. Análise assistida por IA

A função a n a l i s e _ i a ( ) opera como um classificador explicável baseado em regras. Ela utiliza os próprios dados de telemetria e o resultado da verificação para produzir três saídas: classificações, anomalias e sugestões.

- No cenário aprovado, a IA classifica temperatura como nominal, energia como adequada, pressão como nominal e módulos como operacionais.
- No cenário crítico, a IA identifica energia insuficiente, risco de falha de propulsão por pressão baixa, falha em módulos críticos e comprometimento estrutural.
- As sugestões são objetivas e acionáveis: recarregar energia, pressurizar tanques, substituir módulos e adiar a missão.

Categoria	Exemplos produzidos pela função
Classificações	Temperatura interna: NOMINAL; Energia: ADEQUADA; Pressão dos tanques: NOMINAL.
Anomalias	Nível de energia abaixo do mínimo; pressão próxima ao limite inferior; falha em módulos; integridade comprometida.
Sugestões	Ativar refrigeração, recarregar bancos de energia, pressurizar tanques, substituir módulos e adiar a missão.

.

7. Reflexão crítica: ética, impacto social e sustentabilidade

Em um contexto aeroespacial, decidir pela decolagem não é apenas um ato técnico. É também um ato ético. Quando um sistema automatizado aprova ou reprova uma missão, ele influencia segurança humana, consumo de recursos, risco ambiental e custo econômico. Por isso, a transparência da regra de decisão é essencial. O código analisado tem um mérito importante: suas regras são explícitas, legíveis e auditáveis.

Do ponto de vista social, a exploração espacial produz inovação em materiais, telecomunicações, energia, monitoramento remoto e ciência de dados. No entanto, também exige investimentos elevados e levanta debates sobre prioridades coletivas. Refletir sobre impacto social significa reconhecer que tecnologia de ponta deve gerar benefícios além do setor espacial.

Na dimensão da sustentabilidade, a análise energética mostra que eficiência não é detalhe, mas requisito de sobrevivência. Em sistemas embarcados, cada cálculo, cada sensor e cada redundância consomem energia. Desenvolver software eficiente, seguro e justificável é parte da responsabilidade ambiental e técnica de qualquer missão.